



CSS – Day 3

Navigation & Website Layout

Building a Navigation Bar

Navigation bars are essential for website usability. We'll start with a basic unordered list and transform it into a sleek horizontal menu using CSS display properties and styling.

Step 1: HTML

```
<nav>
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#services">Services</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>
```

Step 2: Initial CSS (Remove Bullets)

```
nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
}

nav li {
  display: inline-block;
}
```

Enhancing Your Navigation Bar

Now that we have a horizontal navigation structure, let's add visual appeal with background colors, padding, and interactive hover effects that improve user experience.

Adding Background & Padding

```
nav ul {  
  list-style: none;  
  margin: 0;  
  padding: 0;  
  background-color: #f20374;  
}  
  
nav li {  
  display: inline-block;  
}  
  
nav a {  
  display: block;  
  padding: 15px 20px;  
  color: white;  
  text-decoration: none;  
}
```

Creating Hover

```
nav a:hover {  
  background-color: #d4005e;  
  color: #fff;  
}
```

Creating a Dropdown Menu

Dropdown menus add depth to navigation by revealing submenus on hover. We'll use CSS positioning and display properties to control when dropdown content appears.

HTML Structure with Nested List

```
<nav>
  <ul>
    <li><a href="#home">Home</a></li>
    <li>
      <a href="#products">Products <span>▼</span></a>
      <ul class="dropdown">
        <li><a href="#web">Web Design</a></li>
        <li><a href="#app">App Development</a></li>
        <li><a href="#seo">SEO Services</a></li>
      </ul>
    </li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>
```

CSS Positioning & Display

```
nav .dropdown {
  position: absolute;
  top: 100%;
  left: 0;
  background-color: #f20374;
  list-style: none;
  padding: 0;
  display: none;
  margin: 0;
}

nav li:hover .dropdown {
  display: block;
}

nav .dropdown li {
  display: block;
}
```

📌 **Key Concept:** The `position: relative` on the parent `<li>` and `position: absolute` on the dropdown create a positioning context. The `display: none` hides the menu by default, while `display: block` on hover makes it appear instantly.

Website Layout Structure

Building a complete website layout requires structuring content into logical sections and positioning them using CSS properties like width, display, margin, and padding.

Basic HTML

```
<header>
  <h1>My Website</h1>
  <nav>
    <!-- Navigation items -->
  </nav>
</header>

<main>
  <section id="content">
    <h2>Main Content</h2>
    <p>Page content goes here...</p>
  </section>

  <aside id="sidebar">
    <h3>Sidebar</h3>
    <ul>
      <li><a href="#">Link 1</a></li>
      <li><a href="#">Link 2</a></li>
    </ul>
  </aside>
</main>

<footer>
  <p>© 2024 My Website</p>
```

Applying Layout CSS

```
main {
  display: flex;
  margin: 20px;
}

#content {
  flex: 3;
  padding-right: 20px;
}

#sidebar {
  flex: 1;
  background-color: #f20374;
  padding: 15px;
  color: white;
}
```

Mini Project Task

Time to apply what you've learned! Build your own homepage using the skills from today's workshop. This hands-on practice will solidify your understanding of navigation and layout techniques.



Navbar with Hover

Create a navigation bar with at least 3-4 menu items. Add hover effects that change the background color when users mouse over links.



Dropdown Menu

Include one dropdown menu with 2-3 submenu items. Use the positioning and display techniques we learned to show and hide the dropdown on hover.



Two Content Sections

Structure your main content area with two sections. Use CSS to position them side by side or stacked, depending on your design preference.



Sidebar & Footer

Add a sidebar for supplementary content and a footer with copyright information. Style both with background colors and padding.

Challenge Yourself!

Go beyond the basics by customizing colors, adding borders, adjusting spacing, or experimenting with different layout approaches. Try using different display properties like `grid` instead of `flex` to see how it affects your layout. Don't forget to test your hover effects work smoothly!



Pro Tip: Start with the HTML structure first, then gradually add CSS styles. Test each feature individually before moving to the next. This incremental approach makes debugging much easier.